

PYLER FRAMEWORK

Next-Generation Multi-Runtime MVC Ecosystem
PHP · Python · React · React Native — One Unified Architecture

PHP 8.x

Python 3.11+

React 18

React Native

v1.0 Blueprint

This document is the **complete step-by-step blueprint** for building the Pyler Framework — a unified MVC ecosystem where PHP, Python, React, and React Native all share one routing system, one ORM, one auth layer, one CLI, and one migration system. Every file, folder, command, and code snippet is included.

Feature	Laravel	Django	Pyler
Multi-language runtime	✗	✗	✓
PHP + Python same MVC	✗	✗	✓
Shared ORM across runtimes	✗	✗	✓
React / RN built-in	✗	✗	✓
Single CLI for all runtimes	✗	✗	✓
No separate REST API needed	✗	✗	✓
Database migrations unified	PHP only	Python only	Both

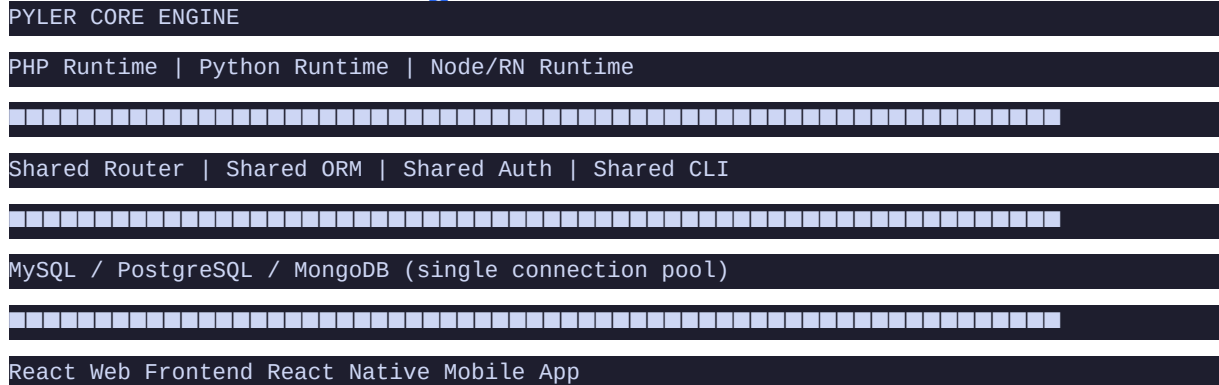
Table of Contents

1. Architecture Overview & Vision
2. Full Directory Structure (with commands)
3. Core Engine — Router, Request, Response
4. Multi-Runtime System (PHP + Python)
5. Shared ORM & Database Layer
6. Migration System (Unified)
7. Auth System (Shared)
8. Middleware System
9. Bridge Layer — PHP ↔ Python IPC
10. Frontend Integration — React & React Native
11. CLI Tool — pyler command
12. Controllers — PHP & Python
13. Models — PHP & Python
14. Views & Template Engine
15. Services & Dependency Injection
16. Validation System
17. API Layer (Internal — No REST needed)
18. Configuration & Environment
19. Full Sample App — Ecommerce
20. Deployment Guide
21. Development Phases & Roadmap

1. Architecture Overview & Vision

Pyler is a **next-generation, multi-runtime MVC framework**. Unlike Laravel (PHP only) or Django (Python only), Pyler lets developers write PHP controllers, Python AI controllers, React web frontends, and React Native mobile apps — all under **one unified architecture**. The developer never needs to configure separate projects or REST APIs between languages.

Core Architecture Diagram



Key Design Principles

- **Single Routing System:** One routes/web.php and routes/api.php handles all runtimes.
- **Runtime Transparency:** Developer writes `Route::get("/users", "UserController@index")` — Pyler decides if PHP or Python handles it.
- **Shared ORM:** `Pyler::model("User")` works identically in PHP and Python, hits the same DB.
- **No REST API Required:** PHP, Python, and React communicate through the internal bridge layer — no HTTP overhead between runtimes.
- **Unified CLI:** `pyler make:controller UserController --runtime=php|python|both`
- **Unified Migrations:** One migration file manages schema for both PHP and Python models.
- **Single Auth System:** JWT / Session auth works across PHP and Python endpoints.
- **Monorepo Structure:** Web and mobile frontend live inside the framework folder.

2. Full Directory Structure

Run these commands to scaffold the entire Pyler project. Every folder is explained below.

Step 1 — Create root project

```
mkdir pyler-app && cd pyler-app
git init
touch .env .gitignore README.md composer.json package.json pyproject.toml
```

Step 2 — Create all directories

```
mkdir -p app/controllers/php
mkdir -p app/controllers/python
mkdir -p app/models/php
mkdir -p app/models/python
mkdir -p app/middleware
mkdir -p app/services/php
mkdir -p app/services/python
mkdir -p app/repositories
mkdir -p app/events
mkdir -p app/listeners
mkdir -p app/jobs
mkdir -p app/views/templates
mkdir -p app/views/components
mkdir -p core/router
mkdir -p core/request
mkdir -p core/response
mkdir -p core/database/php
mkdir -p core/database/python
mkdir -p core/auth
mkdir -p core/validation
mkdir -p core/runtime
mkdir -p core/bridge
mkdir -p core/cache
mkdir -p core/queue
mkdir -p core/events
mkdir -p core/exceptions
mkdir -p frontend/web/src/components
mkdir -p frontend/web/src/pages
mkdir -p frontend/web/src/hooks
mkdir -p frontend/web/src/store
mkdir -p frontend/web/src/api
mkdir -p frontend/mobile/src/components
mkdir -p frontend/mobile/src/screens
mkdir -p frontend/mobile/src/navigation
mkdir -p frontend/mobile/src/store
mkdir -p frontend/mobile/src/api
mkdir -p bridge/ipc
mkdir -p bridge/socket
mkdir -p routes
mkdir -p config
mkdir -p database/migrations/php
mkdir -p database/migrations/python
mkdir -p database/seeder
```

```
mkdir -p database/factories
mkdir -p storage/logs
mkdir -p storage/cache
mkdir -p storage/uploads
mkdir -p storage/sessions
mkdir -p public/assets
mkdir -p cli/commands
mkdir -p tests/php
mkdir -p tests/python
mkdir -p tests/e2e
```

Step 3 — Create all core files

```
# Core engine files
touch core/router/Router.php
touch core/router/router.py
touch core/router/Route.php
touch core/request/Request.php
touch core/request/request.py
touch core/response/Response.php
touch core/response/response.py
touch core/database/php/DB.php
touch core/database/php/QueryBuilder.php
touch core/database/php/Model.php
touch core/database/python/db.py
touch core/database/python/query_builder.py
touch core/database/python/model.py
touch core/auth/Auth.php
touch core/auth/auth.py
touch core/auth/JWTManager.php
touch core/auth/jwt_manager.py
touch core/validation/Validator.php
touch core/validation/validator.py
touch core/runtime/RuntimeLoader.php
touch core/runtime/RuntimeLoader.py
touch core/runtime/PHPRuntime.php
touch core/runtime/PythonRuntime.py
touch core/bridge/Bridge.php
touch core/bridge/bridge.py
touch core/bridge/IPCServer.php
touch core/bridge/ipc_server.py
touch core/cache/Cache.php
touch core/cache/cache.py
touch core/queue/Queue.php
touch core/queue/queue.py
touch core/exceptions/Handler.php
touch core/exceptions/handler.py

# Route files
touch routes/web.php
touch routes/api.php
touch routes/channels.php

# Config files
```

```

touch config/app.php
touch config/database.php
touch config/auth.php
touch config/cache.php
touch config/queue.php
touch config/runtime.php
touch config/frontend.php

# Entry points
touch public/index.php
touch public/.htaccess
touch pyler          # CLI binary

# Bootstrap
touch bootstrap/app.php
touch bootstrap/app.py

```

Full annotated directory tree

```

pyler-app/
├── app/                                # Application code
│   ├── controllers/                   # PHP controllers
│   │   ├── php/
│   │   └── python/                   # Python (AI/ML) controllers
│   ├── models/
│   │   ├── php/                       # Eloquent-style PHP models
│   │   └── python/                   # SQLAlchemy-style Python models
│   ├── middleware/                   # Request filters (auth, cors, rate-limit)
│   ├── services/
│   │   ├── php/                       # PHP business logic services
│   │   └── python/                   # Python ML/AI services
│   ├── repositories/                 # Data access layer
│   ├── events/ & listeners/          # Event-driven architecture
│   ├── jobs/                         # Background jobs / queues
│   └── views/                        # Blade-like templates
├── core/                             # Framework engine (don't edit)
│   ├── router/                       # Unified routing
│   ├── request/ & response/          # HTTP layer
│   ├── database/                     # ORM — PHP + Python
│   ├── auth/                         # JWT + Session auth
│   ├── validation/                   # Input validation
│   ├── runtime/                      # PHP/Python runtime loader
│   ├── bridge/                       # IPC bridge between runtimes
│   ├── cache/ & queue/               # Redis/Memcached, job queues
│   └── exceptions/                  # Error handling
├── frontend/
│   ├── web/                          # React 18 web app
│   │   ├── src/components, pages, hooks, store, api
│   │   └── mobile/                  # React Native app
│   │       ├── src/components, screens, navigation, store, api
├── bridge/                           # Runtime communication layer

```

```

■ ■ ■ ■ ipc/                # Unix socket / named pipe
■ ■ ■ ■ socket/            # WebSocket bridge
■
■ ■ ■ ■ database/
■ ■ ■ ■ migrations/php/    # PHP migration files
■ ■ ■ ■ migrations/python/ # Python migration files (Alembic)
■ ■ ■ ■ seeders/           # Seed data
■ ■ ■ ■ factories/         # Model factories for testing
■
■ ■ ■ ■ routes/
■ ■ ■ ■ web.php            # Web routes (PHP + Python)
■ ■ ■ ■ api.php            # API routes
■
■ ■ ■ ■ config/            # All configuration files
■ ■ ■ ■ storage/           # Logs, cache, uploads, sessions
■ ■ ■ ■ public/            # Web root (index.php)
■ ■ ■ ■ cli/               # pyler CLI commands
■ ■ ■ ■ tests/             # PHP + Python + E2E tests
■ ■ ■ ■ pyler              # CLI entry point

```

3. Core Engine — Router, Request, Response

3.1 Unified Router (routes/web.php)

The router is the heart of Pyler. A single file defines all routes. Each route specifies which runtime handles it (php, python, or auto). If no runtime is specified, Pyler auto-detects based on the controller location.

```
# routes/web.php
```

[illegible]

3.2 Router Core (core/router/Router.php)

```
# core/router/Router.php
```

```
namespace Pyler\Core\Router;

class Router {
    private static array $routes = [];
    private static RuntimeLoader $runtime;

    public static function get(string $uri, string $handler, array $opts=[]): Route {
        return self::addRoute('GET', $uri, $handler, $opts);
    }
}
```

```

    public static function post(string $uri, string $handler, array $opts=[]): Route {
        return self::addRoute('POST', $uri, $handler, $opts);
    }
    public static function put(string $uri, string $handler, array $opts=[]): Route {
        return self::addRoute('PUT', $uri, $handler, $opts);
    }
    public static function delete(string $uri, string $handler, array $opts=[]): Route {
        return self::addRoute('DELETE', $uri, $handler, $opts);
    }
    public static function resource(string $uri, string $ctrl): void {
        self::get($uri,          "$ctrl@index");
        self::post($uri,         "$ctrl@store");
        self::get("$uri/{id}",    "$ctrl@show");
        self::put("$uri/{id}",    "$ctrl@update");
        self::delete("$uri/{id}", "$ctrl@destroy");
    }

    public static function dispatch(Request $request): Response {
        foreach (self::$routes as $route) {
            if ($route->matches($request)) {
                // Determine runtime from handler prefix
                $runtime = $route->getRuntime(); // 'php' | 'python'
                return self::$runtime->dispatch($runtime, $route, $request);
            }
        }
        throw new NotFoundException("Route not found: " . $request->uri());
    }
}

```

3.3 Python Router (core/router/router.py)

core/router/router.py

```

# core/router/router.py
from typing import Callable, Dict, List
from core.request.request import Request
from core.response.response import Response

class PyRoute:
    def __init__(self, method: str, uri: str, handler: Callable):
        self.method = method
        self.uri = uri
        self.handler = handler
        self._middleware: List[str] = []

    def middleware(self, *mw: str) -> 'PyRoute':
        self._middleware.extend(mw)
        return self

class PyRouter:
    _routes: List[PyRoute] = []

    @classmethod
    def get(cls, uri: str, handler: Callable) -> PyRoute:
        r = PyRoute('GET', uri, handler)

```

```

        cls._routes.append(r)
        return r

    @classmethod
    def post(cls, uri: str, handler: Callable) -> PyRoute:
        r = PyRoute('POST', uri, handler)
        cls._routes.append(r)
        return r

    @classmethod
    async def dispatch(cls, request: Request) -> Response:
        for route in cls._routes:
            if route.matches(request):
                for mw_name in route._middleware:
                    mw = MiddlewareRegistry.resolve(mw_name)
                    request = await mw.handle(request)
                return await route.handler(request)
        raise RouteNotFoundException(f"No route: {request.path}")

```

4. Multi-Runtime System

The Runtime Loader is what makes Pyler unique. It reads the route handler prefix (php: or python:) and dispatches to the correct runtime. If no prefix, it checks which folder has the controller.

core/runtime/RuntimeLoader.php

```
// core/runtime/RuntimeLoader.php
namespace Pyler\Core\Runtime;

class RuntimeLoader {
    private PHPRuntime $phpRuntime;
    private PythonRuntime $pyRuntime;

    public function dispatch(string $runtime, Route $route, Request $req): Response {
        return match($runtime) {
            'python' => $this->pyRuntime->handle($route, $req),
            'php'    => $this->phpRuntime->handle($route, $req),
            default  => $this->autoDetect($route, $req),
        };
    }

    private function autoDetect(Route $route, Request $req): Response {
        [$ctrl, $method] = explode('@', $route->handler);
        $phpPath = app_path("controllers/php/{$ctrl}.php");
        $pyPath  = app_path("controllers/python/{$ctrl}.py");

        if (file_exists($phpPath)) return $this->phpRuntime->handle($route, $req);
        if (file_exists($pyPath))  return $this->pyRuntime->handle($route, $req);
        throw new RuntimeException("Controller {$ctrl} not found in php or python");
    }
}
```

core/runtime/PythonRuntime.php

```
// core/runtime/PythonRuntime.php
namespace Pyler\Core\Runtime;

class PythonRuntime {
    private Bridge $bridge;

    public function handle(Route $route, Request $req): Response {
        // Serialize request and send to Python process via IPC bridge
        $payload = [
            'controller' => $route->controller,
            'method'      => $route->method,
            'params'      => $req->all(),
            'headers'     => $req->headers(),
            'user'        => $req->user()?->toArray(),
        ];

        $result = $this->bridge->send('python_dispatch', $payload);
        return Response::fromArray($result);
    }
}
```

config/runtime.php

```
// config/runtime.php
return [
    'php' => [
        'enabled' => true,
        'version' => '8.2',
    ],
    'python' => [
        'enabled' => true,
        'executable' => env('PYTHON_BIN', '/usr/bin/python3'),
        'host' => env('PYTHON_HOST', '127.0.0.1'),
        'port' => env('PYTHON_PORT', 9001),
        'timeout' => 30,
        'workers' => 4,
    ],
    'node' => [
        'enabled' => true,
        'port' => env('NODE_PORT', 3000),
    ],
];
```

5. Shared ORM & Database Layer

Both PHP and Python models connect to the **same database**. The PHP ORM is Eloquent-style. The Python ORM is SQLAlchemy-style. `Pylar::model("User")` resolves to the correct runtime model automatically.

5.1 PHP Model (app/models/php/User.php)

app/models/php/User.php

```
// app/models/php/User.php
namespace App\Models\PHP;

use Pylar\Core\Database\PHP\Model;

class User extends Model {
    protected string $table      = 'users';
    protected array  $fillable   = ['name', 'email', 'password', 'role'];
    protected array  $hidden     = ['password'];
    protected array  $casts      = ['email_verified_at' => 'datetime'];

    // Relationships
    public function posts(): HasMany {
        return $this->hasMany(Post::class);
    }
    public function profile(): HasOne {
        return $this->hasOne(Profile::class);
    }

    // Scopes
    public function scopeActive($query) {
        return $query->where('status', 'active');
    }

    // Accessors
    public function getFullNameAttribute(): string {
        return "{$this->first_name} {$this->last_name}";
    }
}

// Usage:
// User::all();
// User::find(1);
// User::where('role', 'admin')->active()->get();
// User::create(['name' => 'John', 'email' => 'john@example.com']);
```

5.2 Python Model (app/models/python/User.py)

app/models/python/User.py

```
# app/models/python/User.py
from pylar.core.database.python.model import PylarModel
from sqlalchemy import Column, String, Integer, DateTime, Boolean
from sqlalchemy.orm import relationship

class User(PylarModel):
```

```

__tablename__ = 'users' # Same table as PHP model!

id          = Column(Integer, primary_key=True)
name        = Column(String(100), nullable=False)
email       = Column(String(255), unique=True, nullable=False)
password    = Column(String(255))
role        = Column(String(50), default='user')
created_at  = Column(DateTime, server_default='now()')

# Relationships
posts = relationship('Post', back_populates='user')

@classmethod
def active(cls):
    return cls.query.filter_by(status='active')

@classmethod
def find(cls, id: int):
    return cls.query.get(id)

@classmethod
def where(cls, **kwargs):
    return cls.query.filter_by(**kwargs)

# Usage (identical to PHP syntax via PylerModel):
# User.all()
# User.find(1)
# User.where(role='admin').active().all()
# User.create(name='John', email='john@example.com')

```

5.3 DB Config (config/database.php)

```

// config/database.php
return [
    'default' => env('DB_CONNECTION', 'mysql'),
    'connections' => [
        'mysql' => [
            'driver'   => 'mysql',
            'host'     => env('DB_HOST', '127.0.0.1'),
            'port'     => env('DB_PORT', '3306'),
            'database' => env('DB_DATABASE', 'pyler_db'),
            'username' => env('DB_USERNAME', 'root'),
            'password' => env('DB_PASSWORD', ''),
            'charset'  => 'utf8mb4',
            // Both PHP and Python use this same connection
            'shared'   => true,
        ],
        'pgsql' => [
            'driver'   => 'pgsql',
            'host'     => env('DB_HOST', '127.0.0.1'),
            'port'     => env('DB_PORT', '5432'),
            'database' => env('DB_DATABASE', 'pyler_db'),
            'username' => env('DB_USERNAME', 'postgres'),
            'password' => env('DB_PASSWORD', ''),
        ],
    ],
];

```

```
    ],  
    ],  
];
```

6. Migration System (Unified)

Pyler migrations run **once** and create tables for both PHP and Python. You never write separate migration files per language. The migration system uses PHP for schema definition and auto-generates the Python Alembic config.

Create a migration

```
pyler make:migration create_users_table
pyler make:migration create_products_table
pyler make:migration add_role_to_users
```

Migration file (database/migrations/php/2024_01_01_create_users_table.php)

```
use Pyler\Core\Database\Migration;
use Pyler\Core\Database\Schema;
use Pyler\Core\Database\Blueprint;

class CreateUsersTable extends Migration {

    public function up(): void {
        Schema::create('users', function (Blueprint $table) {
            $table->id(); // AUTO INCREMENT PK
            $table->string('name', 100);
            $table->string('email', 255)->unique();
            $table->string('password', 255)->nullable();
            $table->string('role', 50)->default('user');
            $table->boolean('is_active')->default(true);
            $table->string('avatar')->nullable();
            $table->timestamp('email_verified_at')->nullable();
            $table->rememberToken(); // for sessions
            $table->softDeletes(); // deleted_at
            $table->timestamps(); // created_at, updated_at
        });
    }

    public function down(): void {
        Schema::dropIfExists('users');
    }
}
```

Run migrations

```
# Run all pending migrations (PHP + Python alembic auto-sync)
pyler migrate

# Rollback last batch
pyler migrate:rollback

# Reset and re-run all
pyler migrate:fresh

# Seed after migration
pyler migrate:fresh --seed

# Show migration status
```

```
pyler migrate:status
```

Python Alembic auto-generated (database/migrations/python/env.py)

```
# database/migrations/python/env.py
# AUTO-GENERATED by Pyler — do not edit manually
# Run: pyler migrate (handles both PHP and Python)

from alembic import context
from pyler.core.database.python.db import engine, Base
from app.models.python import * # import all Python models

def run_migrations():
    with engine.connect() as connection:
        context.configure(connection=connection, target_metadata=Base.metadata)
        with context.begin_transaction():
            context.run_migrations()
```

7. Auth System (Shared)

Pyler Auth works across PHP and Python routes. JWT tokens generated by a PHP login route are verified by Python routes too — because both read from the same secret and user table.

7.1 Auth Controller (<app/controllers/php/AuthController.php>)

```
namespace App\Controllers\PHP;

use Pyler\Core\Auth\Auth;
use Pyler\Core\Http\Request;
use Pyler\Core\Http\Response;
use App\Models\PHP\User;

class AuthController {

    public function login(Request $request): Response {
        $credentials = $request->validate([
            'email' => 'required|email',
            'password' => 'required|min:6',
        ]);

        if (!Auth::attempt($credentials)) {
            return Response::json(['error' => 'Invalid credentials'], 401);
        }

        $user = Auth::user();
        $token = Auth::token()->generate($user); // JWT

        return Response::json([
            'token' => $token,
            'user' => $user->toArray(),
        ]);
    }

    public function register(Request $request): Response {
        $data = $request->validate([
            'name' => 'required|string|max:100',
            'email' => 'required|email|unique:users',
            'password' => 'required|min:8|confirmed',
        ]);

        $user = User::create([
            ...$data,
            'password' => Auth::hash($data['password']),
        ]);

        $token = Auth::token()->generate($user);

        return Response::json(['token' => $token, 'user' => $user], 201);
    }

    public function me(Request $request): Response {
        return Response::json(Auth::user());
    }
}
```

```

    }

    public function logout(Request $request): Response {
        Auth::logout();
        return Response::json(['message' => 'Logged out']);
    }
}

```

7.2 Middleware — Auth Guard (app/middleware/AuthMiddleware.php)

```

namespace App\Middleware;
use Pylar\Core\Auth\Auth;

class AuthMiddleware {
    public function handle(Request $request, Closure $next): Response {
        $token = $request->bearerToken()
            ?? $request->cookie('pyler_session');

        if (!$token || !Auth::token()->verify($token)) {
            return Response::json(['error' => 'Unauthenticated'], 401);
        }

        $request->setUser(Auth::token()->user($token));
        return $next($request);
    }
}

```

7.3 Python Auth Middleware (app/middleware/auth_middleware.py)

```

# app/middleware/auth_middleware.py
from pylar.core.auth.jwt_manager import JWTManager

class AuthMiddleware:
    async def handle(self, request, next_handler):
        token = request.bearer_token or request.cookie('pyler_session')
        if not token:
            return Response.json({'error': 'Unauthenticated'}, 401)

        user = JWTManager.verify(token)  # same JWT secret as PHP
        if not user:
            return Response.json({'error': 'Invalid token'}, 401)

        request.user = user
        return await next_handler(request)

```

8. Middleware System

```
# Create middleware
pyler make:middleware CorsMiddleware
pyler make:middleware RateLimitMiddleware
pyler make:middleware AdminMiddleware
```

Global Middleware (config/app.php — middleware stack)

```
// config/app.php
'middleware' => [
    'global' => [
        App\Middleware\CorsMiddleware::class,
        App\Middleware\SecurityHeadersMiddleware::class,
        App\Middleware\RateLimitMiddleware::class,
    ],
    'groups' => [
        'web' => [
            App\Middleware\SessionMiddleware::class,
            App\Middleware\CsrfMiddleware::class,
        ],
        'api' => [
            App\Middleware\JsonMiddleware::class,
        ],
        'auth' => [App\Middleware\AuthMiddleware::class],
        'admin' => [App\Middleware\AdminMiddleware::class],
    ],
    'aliases' => [
        'auth' => App\Middleware\AuthMiddleware::class,
        'admin' => App\Middleware\AdminMiddleware::class,
        'throttle' => App\Middleware\RateLimitMiddleware::class,
        'cors' => App\Middleware\CorsMiddleware::class,
        'verified' => App\Middleware>EmailVerifiedMiddleware::class,
    ],
],
```

CORS Middleware (app/middleware/CorsMiddleware.php)

```
namespace App\Middleware;

class CorsMiddleware {
    public function handle(Request $request, Closure $next): Response {
        if ($request->method() === 'OPTIONS') {
            return Response::make('', 204, $this->headers());
        }
        $response = $next($request);
        foreach ($this->headers() as $k => $v) {
            $response->header($k, $v);
        }
        return $response;
    }

    private function headers(): array {
        return [
            'Access-Control-Allow-Origin' => config('cors.origin', '*'),
            'Access-Control-Allow-Methods' => 'GET,POST,PUT,DELETE,OPTIONS',
        ];
    }
}
```

```
        'Access-Control-Allow-Headers' => 'Content-Type,Authorization,X-Requested-With',
    ];
}
```

9. Bridge Layer — PHP ↔ Python IPC

The Bridge is the magic of Pyler. When a PHP controller needs Python AI, or when routing dispatches to Python, the Bridge sends serialized data over a Unix socket (same server) or TCP (multi-server). No HTTP overhead. No REST API needed.

```
# core/bridge/Bridge.php
```

[illegible]

```
# core/bridge/ipc_server.py
```

```
# core/bridge/ipc_server.py
# Started by: pyler serve --python
import asyncio, json, os
from pyler.core.runtime.PythonRuntime import PythonRuntime

SOCKET_PATH = 'storage/bridge/pyler.sock'

class IPCServer:
    def __init__(self):
        self.runtime = PythonRuntime()
```

```

async def handle_client(self, reader, writer):
    data = await reader.readline()
    msg = json.loads(data.decode())

    result = await self.runtime.dispatch(
        msg['action'], msg.get('data', {}))

    writer.write((json.dumps(result) + '\n').encode())
    await writer.drain()
    writer.close()

async def start(self):
    if os.path.exists(SOCKET_PATH):
        os.remove(SOCKET_PATH)
    server = await asyncio.start_unix_server(
        self.handle_client, path=SOCKET_PATH)
    print(f'[Pyler Python IPC] Listening on {SOCKET_PATH}')
    async with server:
        await server.serve_forever()

if __name__ == '__main__':
    asyncio.run(IPCServer().start())

```

Using the Bridge from PHP controller

```

// app/controllers/php/ProductController.php
class ProductController {
    private Bridge $bridge;

    public function analyze(Request $request): Response {
        $product = Product::find($request->input('id'));

        // Call Python AI – no REST API needed!
        $analysis = $this->bridge->callPython(
            'AIController',
            'analyzeProduct',
            ['product' => $product->toArray()]
        );

        return Response::json([
            'product' => $product,
            'analysis' => $analysis,
        ]);
    }
}

```

10. Frontend Integration — React & React Native

Setup React Web Frontend

```
cd frontend/web
npx create-react-app . --template typescript
# or
npm create vite@latest . -- --template react-ts

npm install axios react-router-dom zustand @tanstack/react-query
```

frontend/web/src/api/client.ts

```
// frontend/web/src/api/client.ts
import axios from 'axios';

const client = axios.create({
  baseURL: process.env.REACT_APP_API_URL || 'http://localhost:8000',
  headers: { 'Content-Type': 'application/json' },
});

// Auto-attach JWT token
client.interceptors.request.use(config => {
  const token = localStorage.getItem('pyler_token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Auto-refresh on 401
client.interceptors.response.use(
  res => res,
  async err => {
    if (err.response?.status === 401) {
      localStorage.removeItem('pyler_token');
      window.location.href = '/login';
    }
    return Promise.reject(err);
  }
);

export default client;
```

frontend/web/src/api/users.ts

```
// frontend/web/src/api/users.ts
import client from './client';

export const userApi = {
  getAll: () => client.get('/users'),
  getOne: (id) => client.get(`/users/${id}`),
  create: (data) => client.post('/users', data),
  update: (id, d) => client.put(`/users/${id}`, d),
  delete: (id) => client.delete(`/users/${id}`),
  me: () => client.get('/auth/me'),
};
```

```
export const aiApi = {
  sentiment: (text) => client.get('/ai/sentiment', { params: { text } }),
  predict: (data) => client.post('/ai/predict', data),
};
```

React Native Setup (frontend/mobile)

```
cd frontend/mobile
npx react-native init PylerMobile --template react-native-template-typescript

npm install @react-navigation/native @react-navigation/stack
npm install axios zustand @tanstack/react-query
npm install react-native-async-storage/async-storage
```

frontend/mobile/src/api/client.ts

```
// frontend/mobile/src/api/client.ts
// Same pattern as web – just uses AsyncStorage for token
import axios from 'axios';
import AsyncStorage from '@react-native-async-storage/async-storage';

const client = axios.create({
  baseURL: 'http://your-server.com', // or local IP in dev
});

client.interceptors.request.use(async config => {
  const token = await AsyncStorage.getItem('pyler_token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

export default client;
```

config/frontend.php

```
// config/frontend.php
return [
  'web' => [
    'path' => base_path('frontend/web'),
    'build' => base_path('frontend/web/build'),
    'dev_url' => env('REACT_DEV_URL', 'http://localhost:3000'),
  ],
  'mobile' => [
    'path' => base_path('frontend/mobile'),
    'bundler' => 'metro',
    'platforms' => ['ios', 'android'],
  ],
  'ssr' => env('FRONTEND_SSR', false),
];
```

11. CLI Tool — pyler command

All CLI Commands Reference

Command	Description
<code>pyler new my-app</code>	Create a new Pyler project
<code>pyler serve</code>	Start PHP + Python + React dev servers
<code>pyler serve --php-only</code>	Start only PHP server
<code>pyler serve --python-only</code>	Start only Python IPC server
<code>pyler make:controller UserController</code>	Create PHP controller
<code>pyler make:controller AIController --runtime=python</code>	Create Python controller
<code>pyler make:model User</code>	Create PHP + Python model pair
<code>pyler make:model User --php</code>	PHP model only
<code>pyler make:model User --python</code>	Python model only
<code>pyler make:migration create_users_table</code>	Create migration
<code>pyler migrate</code>	Run all pending migrations
<code>pyler migrate:rollback</code>	Rollback last batch
<code>pyler migrate:fresh --seed</code>	Reset DB and seed
<code>pyler make:middleware AuthMiddleware</code>	Create middleware
<code>pyler make:service PaymentService</code>	Create service class
<code>pyler make:seeder UserSeeder</code>	Create database seeder
<code>pyler db:seed</code>	Run all seeders
<code>pyler db:seed --class=UserSeeder</code>	Run specific seeder
<code>pyler cache:clear</code>	Clear all caches
<code>pyler route:list</code>	List all registered routes
<code>pyler test</code>	Run PHP + Python tests
<code>pyler test --php</code>	Run PHP tests only
<code>pyler test --python</code>	Run Python tests only
<code>pyler build</code>	Build React web for production
<code>pyler build:mobile ios</code>	Build React Native iOS
<code>pyler build:mobile android</code>	Build React Native Android
<code>pyler tinker</code>	Interactive PHP REPL with app loaded
<code>pyler shell:python</code>	Interactive Python REPL with app loaded
<code>pyler env:check</code>	Verify environment setup

CLI Binary (pyler)

```
#!/usr/bin/env php
// pyler - place in project root, chmod +x pyler
define('PYLER_ROOT', __DIR__);
require PYLER_ROOT . '/vendor/autoload.php';

use Pyler\Core\CLI\Application;

$app = new Application('Pyler', '1.0.0');
$app->registerCommands(glob(PYLER_ROOT . '/cli/commands/*.php'));
$app->run($argv);
```

12. Controllers — PHP & Python

PHP Controller (app/controllers/php/UserController.php)

```
namespace App\Controllers\PHP;

use Pylar\Core\Http\Request;
use Pylar\Core\Http\Response;
use App\Models\PHP\User;
use App\Services\PHP\UserService;

class UserController {
    public function __construct(private UserService $userService) {}

    public function index(Request $request): Response {
        $users = User::paginate($request->input('per_page', 15));
        return Response::json($users);
    }

    public function show(Request $request, int $id): Response {
        $user = User::findOrFail($id);
        return Response::json($user);
    }

    public function store(Request $request): Response {
        $data = $request->validate([
            'name' => 'required|string|max:100',
            'email' => 'required|email|unique:users',
            'password' => 'required|min:8',
        ]);
        $user = $this->userService->create($data);
        return Response::json($user, 201);
    }

    public function update(Request $request, int $id): Response {
        $user = User::findOrFail($id);
        $data = $request->validate([
            'name' => 'sometimes|string|max:100',
            'email' => "sometimes|email|unique:users,email,{ $id }",
        ]);
        $user->update($data);
        return Response::json($user);
    }

    public function destroy(int $id): Response {
        User::findOrFail($id)->delete();
        return Response::json(['message' => 'Deleted'], 204);
    }
}
```

Python Controller (app/controllers/python/AIController.py)

```
# app/controllers/python/AIController.py
from pylar.core.http.request import Request
```

```

from pyler.core.http.response import Response
from app.services.python.sentiment_service import SentimentService
from app.services.python.ml_service import MLService
from app.models.python.User import User

class AIController:
    def __init__(self):
        self.sentiment = SentimentService()
        self.ml = MLService()

    async def sentiment(self, request: Request) -> Response:
        text = request.query('text', '')
        result = await self.sentiment.analyze(text)
        return Response.json({'sentiment': result, 'text': text})

    async def predict(self, request: Request) -> Response:
        data = request.validated({
            'features': 'required|list',
            'model': 'sometimes|string',
        })
        prediction = await self.ml.predict(data['features'])
        return Response.json({'prediction': prediction})

    async def analyze_product(self, request: Request) -> Response:
        product = request.input('product')
        analysis = await self.ml.analyze(product)
        # Return to PHP bridge caller
        return Response.json({
            'score': analysis.score,
            'tags': analysis.tags,
            'description': analysis.description,
        })

```

13. Services & Validation

PHP Service (app/services/php/UserService.php)

```
namespace App\Services\PHP;

use App\Models\PHP\User;
use Pyler\Core\Auth\Auth;
use Pyler\Core\Events\EventDispatcher;

class UserService {
    public function create(array $data): User {
        $data['password'] = Auth::hash($data['password']);
        $user = User::create($data);
        EventDispatcher::dispatch('user.created', $user);
        return $user;
    }

    public function update(User $user, array $data): User {
        if (isset($data['password'])) {
            $data['password'] = Auth::hash($data['password']);
        }
        $user->update($data);
        return $user->fresh();
    }
}
```

Python Service (app/services/python/sentiment_service.py)

```
# app/services/python/sentiment_service.py
from transformers import pipeline
from functools import lru_cache

class SentimentService:
    @lru_cache(maxsize=1)
    def _model(self):
        return pipeline('sentiment-analysis')

    async def analyze(self, text: str) -> dict:
        result = self._model()(text)[0]
        return {
            'label': result['label'],
            'score': round(result['score'], 4),
        }
```

Validation (PHP) — app/controllers/php/

```
// Available validation rules in Pyler:
$request->validate([
    'name'      => 'required|string|min:2|max:100',
    'email'     => 'required|email|unique:users',
    'age'       => 'required|integer|min:18|max:120',
    'role'      => 'required|in:admin,user,moderator',
    'password' => 'required|min:8|confirmed',    // needs password_confirmation
    'photo'     => 'sometimes|image|max:2048',    // 2MB max
])
```

```
'tags'      => 'array|max:10',
'tags.*'    => 'string|max:50',
'price'     => 'required|numeric|min:0',
'url'       => 'sometimes|url',
'date'      => 'required|date|after:today',
]);
```

Validation (Python)

```
# Python validation – same rules, Python syntax
data = request.validated({
    'name': 'required|string|min:2|max:100',
    'email': 'required|email|unique:users',
    'age': 'required|integer|min:18',
    'role': 'required|in:admin,user,moderator',
})
```

14. API Layer — No REST API Needed

Traditional setups require a REST API between PHP backend, Python backend, and React frontend. Pyler eliminates this with two mechanisms:

- **Internal Bridge:** PHP ↔ Python communicate via Unix socket (zero HTTP overhead).
- **Pyler Client SDK:** React/RN use a typed client that directly maps to controller methods — no manual REST endpoint writing.
- **Auto API Generation:** `pyler make:api UserController` auto-generates typed TypeScript client for frontend.

Auto-generated TypeScript client (frontend/web/src/api/generated/users.ts)

```
// AUTO-GENERATED by: pyler make:api UserController
// Do not edit – re-run command to regenerate

import client from '../client';

export interface User {
  id: number; name: string; email: string; role: string;
  created_at: string; updated_at: string;
}

export interface PaginatedUsers {
  data: User[]; current_page: number; last_page: number;
  per_page: number; total: number;
}

export const Users = {
  index: (params?: { per_page?: number; page?: number }) =>
    client.get('/users', { params }),

  show: (id: number) =>
    client.get(`/users/${id}`),

  create: (data: Pick & { password: string }) =>
    client.post('/users', data),

  update: (id: number, data: Partial<>) =>
    client.put(`/users/${id}`, data),

  destroy: (id: number) =>
    client.delete(`/users/${id}`),
};
```

Usage in React component

```
// frontend/web/src/pages/Users.tsx
import { Users } from '../../api/generated/users';
import { useQuery } from '@tanstack/react-query';

export default function UsersPage() {
  const { data, isLoading } = useQuery({
    queryKey: ['users'],
```

```
    queryFn: () => Users.index({ per_page: 20 }),
  });

  if (isLoading) return Loading...;

  return (
    {data?.data.data.map(user => (
      {user.name} — {user.email}
    ))}
  );
}
```

15. Configuration & Environment

.env file

[illegible]

16. Full Sample App — Ecommerce Project

Bootstrap with one command:

```
pyler new my-shop
```

Then scaffold everything:

```
# Models (PHP + Python pair)
pyler make:model Product
pyler make:model Order
pyler make:model Category
pyler make:model Cart

# Controllers
pyler make:controller ProductController
pyler make:controller OrderController
pyler make:controller CartController
pyler make:controller AIController --runtime=python
pyler make:controller RecommendController --runtime=python

# Migrations
pyler make:migration create_products_table
pyler make:migration create_orders_table
pyler make:migration create_order_items_table
pyler make:migration create_categories_table
pyler make:migration create_carts_table

# Services
pyler make:service PaymentService
pyler make:service InventoryService
pyler make:service NotificationService

# Middleware
pyler make:middleware AuthMiddleware
pyler make:middleware AdminMiddleware
pyler make:middleware RateLimitMiddleware

# Seeders
pyler make:seeder ProductSeeder
pyler make:seeder CategorySeeder
pyler make:seeder UserSeeder

# Run migrations and seed
pyler migrate --seed

# Generate TypeScript API clients for React
pyler make:api ProductController
pyler make:api OrderController

# Start all servers
pyler serve
```

routes/web.php for ecommerce

```

use Pyler\Core\Router\Route;

// Public
Route::get('/products',          'ProductController@index');
Route::get('/products/{id}',     'ProductController@show');
Route::get('/categories',       'CategoryController@index');

// AI/Python routes
Route::get('/ai/recommend',      'python:RecommendController@index');
Route::post('/ai/search',        'python:SearchController@semantic');

// Auth required
Route::group(['middleware' => 'auth'], function() {
    Route::post('/cart',          'CartController@add');
    Route::get('/cart',          'CartController@index');
    Route::post('/orders',        'OrderController@store');
    Route::get('/orders',         'OrderController@index');
    Route::get('/orders/{id}',    'OrderController@show');
});

// Admin only
Route::group(['prefix'=>'admin', 'middleware'=>['auth', 'admin']], function() {
    Route::resource('products',   'Admin\ProductController');
    Route::resource('orders',     'Admin\OrderController');
    Route::resource('users',      'Admin\UserController');
});

```

17. Deployment Guide

Requirements

Software	Minimum Version	Notes
PHP	8.2+	With ext-pdo, ext-json, ext-sockets
Python	3.11+	With pip, virtualenv
Node.js	20+	For React build
MySQL / PostgreSQL	8.0 / 15+	Or MongoDB 6+
Redis	7+	Cache + Queue
Nginx	1.24+	Or Apache 2.4+
Composer	2.6+	PHP dependency manager

Install dependencies

```
# PHP dependencies
composer install --no-dev --optimize-autoloader

# Python dependencies
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# Node / React dependencies
cd frontend/web && npm install && npm run build
cd ../mobile && npm install

# Environment
cp .env.example .env
php pyler env:check
```

Nginx config

```
server {
    listen 80;
    server_name yourdomain.com;
    root /var/www/pyler-app/public;
    index index.php;

    # PHP
    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }
    location ~ \.php$ {
        fastcgi_pass unix:/run/php/php8.2-fpm.sock;
        include fastcgi_params;
        fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
    }
}
```

```
# React build (served as static)
location /app {
    alias /var/www/pyler-app/frontend/web/build;
    try_files $uri /app/index.html;
}
}
```

Systemd service for Python IPC server

```
# /etc/systemd/system/pyler-python.service
[Unit]
Description=Pyler Python IPC Server
After=network.target

[Service]
User=www-data
WorkingDirectory=/var/www/pyler-app
ExecStart=/var/www/pyler-app/.venv/bin/python core/bridge/ipc_server.py
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target

# Enable and start:
# systemctl enable pyler-python && systemctl start pyler-python
```

18. Development Phases & Roadmap

PHASE 1: Shared Core

- ✓ Router (PHP + Python)
- ✓ Request / Response layer
- ✓ Middleware system
- ✓ Validation engine
- ✓ Auth + JWT
- ✓ Exception handling

PHASE 2: Runtime System

- ✓ PHPRuntime class
- ✓ PythonRuntime class
- ✓ RuntimeLoader (auto-detect)
- ✓ IPC Bridge (Unix socket)
- ✓ Request serialization

PHASE 3: Shared ORM & Migrations

- ✓ PHP Model (Eloquent-style)
- ✓ Python Model (SQLAlchemy-style)
- ✓ Unified migration system
- ✓ Query Builder (both)
- ✓ Relationships
- ✓ Soft deletes
- ✓ Seeders + Factories

PHASE 4: Frontend Integration

- ✓ React web setup + Vite
- ✓ API client auto-generator
- ✓ React Native setup
- ✓ Shared state store (Zustand)
- ✓ React Query integration

PHASE 5: CLI Tool

- ✓ pyler new project
- ✓ make:controller / model / migration
- ✓ migrate / migrate:rollback
- ✓ make:api (TypeScript client)
- ✓ serve (all runtimes)

- ✓ test (PHP + Python)

PHASE 6: Production Features

- ✓ Queue system (Redis)
- ✓ Cache layer
- ✓ Event + Listener system
- ✓ WebSocket support
- ✓ Multi-tenancy
- ✓ Rate limiting

■ PYLER FRAMEWORK

Build what others said was impossible.
PHP + Python + React + React Native — One MVC. One CLI. One Migration. One Auth.

This document covers the complete architecture, all files, all commands, and all code needed to build Pyler from scratch. Start with Phase 1 — build the shared core — and expand from there. Good luck!

